

Chaîne immuable d'annonces académiques

Assistant Académique Décentralisé

DApp blockchain × Intelligence Artificielle × RAG

Réalisé par : Hasna Daoui et Nana DIAWARA

Encadré par : Pr. Mehdi TMIMI

Filière : EIDIA / Cybersécurité

Module : Blockchain & Applications Décentralisées

Résumé

Ce rapport présente la conception et l'implémentation d'un **Assistant Académique Décentralisé**, une application décentralisée (DApp) combinant la technologie blockchain avec l'intelligence artificielle pour moderniser et sécuriser les communications académiques.

La solution repose sur quatre **smart contracts** déployés sur Ethereum (réseau local Hardhat) : **RoleManager** pour la gestion des rôles et des groupes, **AnnouncementLog** pour la publication immuable d'annonces, **AcknowledgmentLog** pour les accusés de réception cryptographiques, et **DocumentRegistry** pour l'intégrité des documents via hachage SHA-256.

Un **agent conversationnel** exploitant la technique RAG (Retrieval-Augmented Generation) et le modèle de langage Mistral AI permet aux étudiants d'interroger le système en langage naturel, avec des réponses contextualisées filtrées par groupe. L'interface est accessible via une application web avec authentification MetaMask, ainsi que via un **bot Telegram** pour les notifications mobiles.

Mots-clés : Blockchain, Smart Contracts, Solidity, Hardhat, DApp, RAG, Mistral AI, MetaMask, Telegram Bot, Ethereum, IPFS.

Abstract — This report presents the design and implementation of a Decentralised Academic Assistant, a DApp combining blockchain technology with artificial intelligence to modernise and secure academic communications. The solution relies on four smart contracts deployed on Ethereum, a RAG-powered conversational agent using Mistral AI, a MetaMask-authenticated web interface, and a Telegram bot for mobile notifications.

Table des matières

Résumé	I
1 Introduction générale	1
1.1 Contexte	1
1.2 Problématique	1
1.3 Objectifs du projet	1
1.4 Structure du rapport	2
2 État de l’art	3
2.1 La blockchain	3
2.2 Les smart contracts	3
2.3 Les DApps	4
2.4 RAG — Retrieval-Augmented Generation	4
2.5 L’IA conversationnelle	4
3 Analyse des besoins	5
3.1 Acteurs du système	5
3.2 Besoins fonctionnels	5
3.3 Besoins non fonctionnels	6
4 Technologies utilisées	7
5 Conception du système	8
5.1 Architecture générale	8
5.2 Architecture des smart contracts	8
5.2.1 RoleManager	9
5.2.2 AnnouncementLog	9
5.2.3 AcknowledgmentLog	9
5.2.4 DocumentRegistry	9
5.3 Architecture RAG	10
6 Implémentation	11
6.1 Smart Contracts	11
6.1.1 RoleManager — Gestion des identités	11

6.1.2	AnnouncementLog — Publication immuable	12
6.1.3	AcknowledgmentLog — Preuve de lecture	12
6.1.4	DocumentRegistry — Intégrité des fichiers	13
6.2	Backend Node.js	13
6.2.1	Service Blockchain (Viem)	13
6.2.2	Middleware d'authentification	14
6.2.3	Pipeline RAG (Mistral AI)	14
6.3	Bot Telegram	15
6.4	Déploiement des contrats	15
7	Interfaces utilisateur	17
7.1	Interface enseignant	17
7.2	Interface étudiant	18
7.3	Chatbot IA	19
7.4	Confirmation MetaMask	20
7.5	Bot Telegram	20
8	Sécurité	21
8.1	Immuabilité blockchain	21
8.2	Intégrité des documents	21
8.3	Contrôle d'accès	21
8.4	Non-répudiation	21
8.5	Authentification MetaMask	21
9	Tests et validation	22
9.1	Tests des smart contracts	22
9.2	Tests backend	23
9.3	Résultats	23
10	Difficultés rencontrées	24
11	Perspectives d'amélioration	25
11.1	Stockage décentralisé avec IPFS	25
11.2	Application mobile native	25
11.3	Déploiement multi-universités	25
11.4	DAO universitaire	25
11.5	Déploiement sur testnet public	25
12	Conclusion générale	26

Table des figures

2.1	Structure d'une blockchain — chaîne de blocs liés par hachage	3
2.2	Pipeline RAG — Retrieval-Augmented Generation	4
5.1	Architecture générale de la DApp en trois couches	8
5.2	Dépendances entre les smart contracts	8
5.3	Architecture détaillée du pipeline RAG	10
6.1	Flux d'exécution du pipeline RAG intégré au backend	15
7.1	Dashboard enseignant — Publication d'annonces et gestion des documents	17
7.2	Dashboard enseignant — Publication des documents	18
7.3	Dashboard étudiant — Consultation des annonces de son groupe	18
7.4	Dashboard étudiant — Vérification des documents de son groupe . . .	19
7.5	Interface chatbot — Réponse contextuelle basée sur les annonces de l'étudiant	19
7.6	Fenêtre de confirmation MetaMask lors d'une publication d'annonce . .	20
7.7	Interaction avec le bot Telegram — Question et réponse IA	20

Chapitre 1

Introduction générale

1.1 Contexte

Les établissements d'enseignement supérieur font face à un défi croissant dans la gestion de leurs communications internes. Les annonces académiques — dates d'examens, remises de devoirs, changements de calendrier — sont souvent dispersées sur plusieurs canaux (email, portails web, groupes de messagerie), ce qui génère confusion, perte d'information et impossibilité de prouver qu'un étudiant a bien reçu une information critique.

Parallèlement, l'essor de la **technologie blockchain** offre une solution inédite : garantir l'immuabilité, la traçabilité et la non-répudiation des données sans recours à une autorité centrale. Combinée aux avancées récentes de l'**intelligence artificielle générative**, cette technologie ouvre la voie à des systèmes d'information académique de nouvelle génération.

1.2 Problématique

Comment garantir que les informations académiques critiques soient :

- **Authentiques** — publiées uniquement par des enseignants accrédités ?
- **Immuables** — impossibles à modifier ou supprimer après publication ?
- **Traçables** — avec preuve cryptographique que chaque étudiant en a pris connaissance ?
- **Accessibles** — via une interface intuitive, y compris depuis un téléphone mobile ?

1.3 Objectifs du projet

Ce projet vise à concevoir et implémenter une DApp académique répondant aux besoins suivants :

1. Publier des annonces académiques sur la blockchain (immutabilité garantie)
2. Gérer les rôles (enseignant, étudiant, administrateur) de façon décentralisée
3. Enregistrer les accusés de réception cryptographiques des étudiants
4. Fournir un chatbot IA capable de répondre aux questions académiques
5. Permettre l'accès via web (MetaMask) et mobile (Telegram)

1.4 Structure du rapport

Ce rapport s'organise comme suit : après un état de l'art des technologies mobilisées, nous présentons l'analyse des besoins, puis la conception architecturale du système. Les chapitres suivants détaillent l'implémentation des smart contracts, du backend, et des interfaces utilisateur. Enfin, nous discutons des résultats des tests et des perspectives d'amélioration.

Chapitre 2

État de l'art

2.1 La blockchain

La blockchain est une structure de données distribuée et décentralisée, conceptualisée par Satoshi Nakamoto en 2008 dans le cadre du protocole Bitcoin [?]. Elle se présente comme une chaîne de blocs liés cryptographiquement, chaque bloc contenant un ensemble de transactions, un horodatage, et le hachage du bloc précédent.

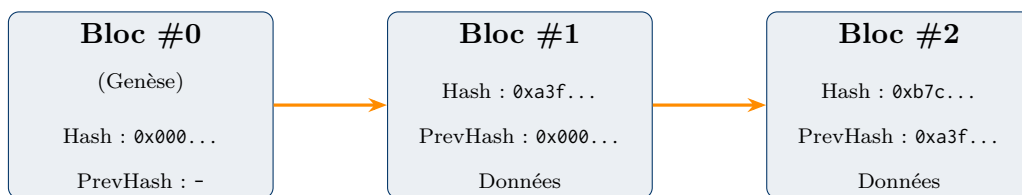


FIGURE 2.1 – Structure d'une blockchain — chaîne de blocs liés par hachage

Les propriétés fondamentales de la blockchain sont :

- **Décentralisation** : aucun nœud central de contrôle
- **Immuabilité** : modifier un bloc invalide tous les blocs suivants
- **Transparence** : toutes les transactions sont vérifiables publiquement
- **Résistance à la censure** : aucune entité ne peut supprimer des données

2.2 Les smart contracts

Introduits par Nick Szabo en 1994 et popularisés par la plateforme **Ethereum** (2015), les smart contracts sont des programmes autonomes s'exécutant sur la blockchain [?]. Ils permettent d'encoder des règles métier directement dans le code, garantissant leur exécution sans intermédiaire.

Définition — Smart Contract

Un smart contract est un programme stocké sur la blockchain, dont l'exécution est déclenchée automatiquement lorsque des conditions prédéfinies sont remplies.

Son code et son état sont publics, vérifiables et immuables.

2.3 Les DApps

Une **DApp** (Decentralised Application) est une application dont la logique métier est implémentée sous forme de smart contracts sur une blockchain. Contrairement aux applications centralisées, les DApps ne dépendent d'aucun serveur central pour leur couche de confiance.

2.4 RAG — Retrieval-Augmented Generation

Le RAG est une technique d'IA qui augmente les capacités d'un LLM (Large Language Model) en lui fournissant, au moment de la génération, des documents pertinents récupérés depuis une base de connaissances [?]. Le processus se déroule en trois étapes :

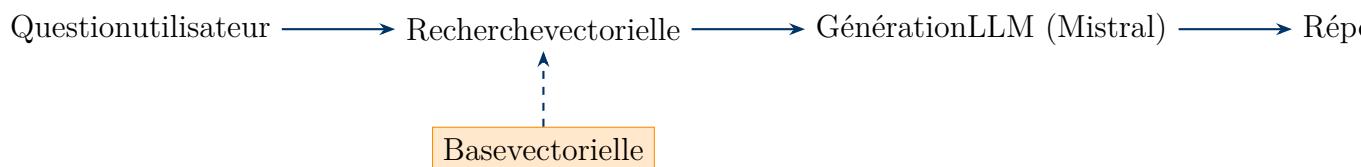


FIGURE 2.2 – Pipeline RAG — Retrieval-Augmented Generation

2.5 L'IA conversationnelle

Les modèles de langage de grande taille (LLM) comme **Mistral AI** permettent de comprendre et générer du texte en langage naturel avec une précision remarquable. Dans notre contexte académique, le modèle `mistral-small-latest` est utilisé pour générer des réponses personnalisées aux questions des étudiants, basées sur les annonces publiées sur la blockchain.

Chapitre 3

Analyse des besoins

3.1 Acteurs du système

Le système implique trois catégories d'acteurs :

Acteur	Description	Permissions blockchain
Administrateur	Responsable de la plateforme	Enregistrement de groupes, assignation de rôles
Enseignant	Publie les annonces et documents	Publication on-chain (rôle PROFESSOR)
Étudiant	Consomme les informations	Accusé de réception (rôle STUDENT)

TABLE 3.1 – Acteurs du système et leurs rôles blockchain

3.2 Besoins fonctionnels

- BF1.** L'enseignant doit pouvoir publier une annonce avec son contenu, sa catégorie et le groupe cible.
- BF2.** L'étudiant doit pouvoir consulter les annonces de son groupe.
- BF3.** L'étudiant doit pouvoir signer un accusé de réception cryptographique pour chaque annonce.
- BF4.** L'enseignant doit pouvoir enregistrer le hash SHA-256 d'un document sur la blockchain.
- BF5.** Tout utilisateur doit pouvoir vérifier l'intégrité d'un document ou d'une annonce.
- BF6.** L'étudiant doit pouvoir interroger le chatbot IA en langage naturel.
- BF7.** L'étudiant doit pouvoir recevoir des notifications via Telegram.
- BF8.** L'administrateur doit pouvoir créer des groupes et assigner des rôles.

3.3 Besoins non fonctionnels

- BNF1. Sécurité** : Authentification par signature cryptographique MetaMask (EIP-712).
- BNF2. Immuabilité** : Les données publiées sur la blockchain ne peuvent être ni modifiées ni supprimées.
- BNF3. Performance** : Temps de réponse du chatbot inférieur à 3 secondes.
- BNF4. Disponibilité** : Le backend doit être disponible 24h/24.
- BNF5. Extensibilité** : Architecture modulaire permettant l'ajout de nouveaux contrats.
- BNF6. Interopérabilité** : Compatible avec tout wallet Ethereum standard.

Chapitre 4

Technologies utilisées

Le tableau suivant récapitule les technologies mobilisées dans ce projet, organisées par couche applicative.

Technologie	Couche	Rôle
Solidity 0.8.20	Smart Contracts	Langage de programmation des contrats Ethereum
Hardhat v2.28	Développement	Environnement local, compilation, déploiement, tests
OpenZeppelin	Smart Contracts	Bibliothèque de contrats sécurisés
Viem v2	Backend	Client TypeScript type-safe pour Ethereum
Node.js	Backend	Runtime JavaScript côté serveur
Express v5	Backend	Framework API RESTful
Mistral AI	IA	LLM pour embeddings (mistral-embed) et génération (mistral-small-latest)
MetaMask	Frontend	Wallet Ethereum, authentification et signature
Telegraf	Mobile	Framework bot Telegram
TypeScript	Global	Typage statique sur l'ensemble du projet

TABLE 4.1 – Technologies utilisées par couche applicative

Chapitre 5

Conception du système

5.1 Architecture générale

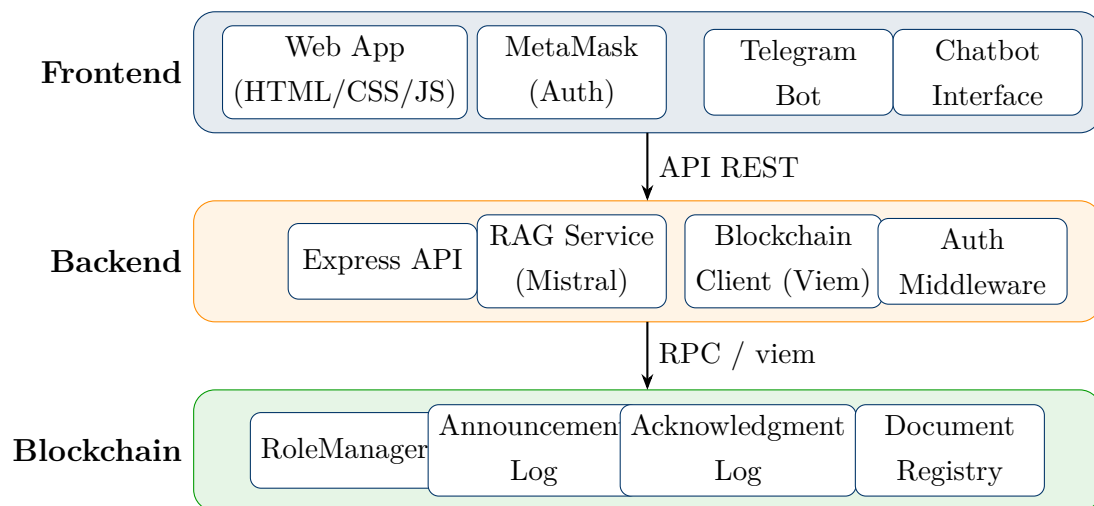


FIGURE 5.1 – Architecture générale de la DApp en trois couches

5.2 Architecture des smart contracts

Les quatre smart contracts forment un écosystème cohérent avec des dépendances claires :

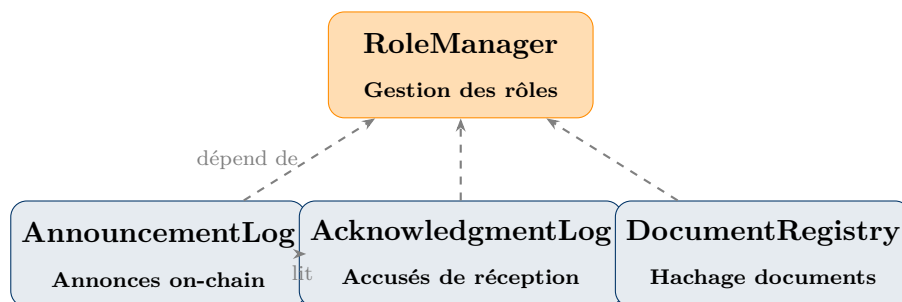


FIGURE 5.2 – Dépendances entre les smart contracts

5.2.1 RoleManager

Contrat central gérant les identités et permissions. Il stocke les rôles (PROFESSOR, STUDENT) et les groupes (MF1, MF2...) sous forme de `bytes32` pour optimiser le coût en gas.

5.2.2 AnnouncementLog

Contrat de publication des annonces. Chaque annonce contient :

- `contentHash` : hachage SHA-256 du contenu
- `group` : identifiant du groupe cible (`bytes32`)
- `category` : catégorie (exam, homework, schedule...)
- `timestamp` : horodatage de publication
- `publisher` : adresse de l'enseignant

5.2.3 AcknowledgmentLog

Contrat d'accusés de réception. Il vérifie que l'étudiant appartient au groupe ciblé par l'annonce et enregistre la preuve cryptographique de la lecture.

5.2.4 DocumentRegistry

Contrat d'enregistrement de documents. Il stocke le hachage SHA-256 des fichiers uploadés, permettant à tout utilisateur de vérifier l'intégrité d'un document sans stocker le fichier lui-même on-chain.

5.3 Architecture RAG

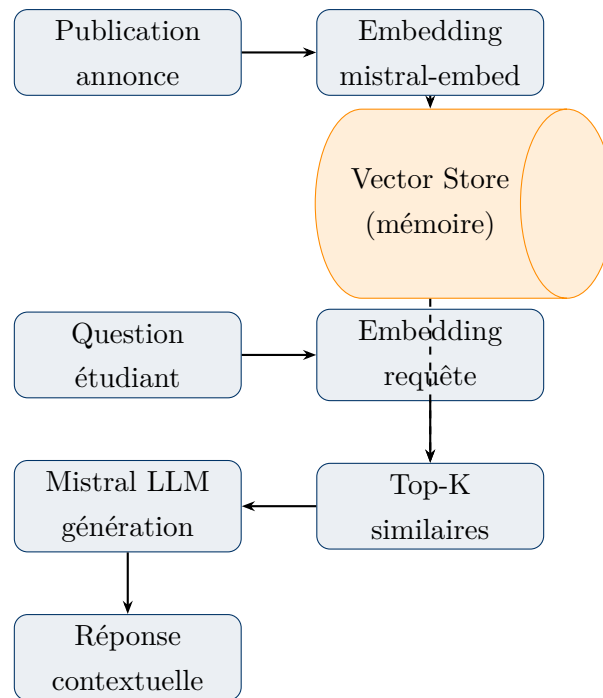


FIGURE 5.3 – Architecture détaillée du pipeline RAG

Chapitre 6

Implémentation

Ce chapitre présente les choix d'implémentation clés du projet, organisés par couche : smart contracts, backend Node.js, et interfaces utilisateur.

6.1 Smart Contracts

Le système repose sur quatre contrats Solidity déployés sur Ethereum (réseau local Hardhat). Leur architecture suit le principe de **séparation des responsabilités** : chaque contrat gère un domaine précis et délègue la vérification des droits au contrat central RoleManager.

6.1.1 RoleManager — Gestion des identités

RoleManager hérite du module `AccessControl` d'OpenZeppelin, qui fournit une implémentation éprouvée du contrôle d'accès basé sur les rôles (RBAC). Trois rôles sont définis : `DEFAULT_ADMIN_ROLE`, `PROFESSOR_ROLE` et `STUDENT_ROLE`, tous stockés en `bytes32` pour minimiser le coût en gas.

Les groupes (MF1, MF2. . .) sont également stockés sous forme de hachage `keccak256` de leur nom, garantissant des comparaisons en $O(1)$ sur la chaîne.

Choix technique — `bytes32` vs `string`

Stocker les identifiants de groupe en `bytes32` plutôt qu'en `string` réduit le coût de stockage d'environ 40% et permet des comparaisons directes (`==`) sans hachage supplémentaire à l'exécution.

Listing 6.1 – RoleManager.sol — Constantes et enregistrement de groupe

```
bytes32 public constant PROFESSOR_ROLE = keccak256("PROFESSOR");
bytes32 public constant STUDENT_ROLE  = keccak256("STUDENT");
bytes32 public constant GROUP_ALL      = keccak256("all");

function registerGroup(string calldata name)
    external onlyRole(DEFAULT_ADMIN_ROLE)
```



```
{
    bytes32 key = keccak256(bytes(name));
    require(!validGroups[key], "RoleManager: group already exists");
    validGroups[key] = true;
    emit GroupRegistered(key, name);
}
```

6.1.2 AnnouncementLog — Publication immuable

Ce contrat stocke chaque annonce sous forme d’une structure `Announcement` contenant le hachage SHA-256 du contenu, le groupe cible, la catégorie, l’horodatage et l’adresse de l’enseignant. Le contenu textuel n’est **jamais** stocké on-chain : seul son empreinte cryptographique y réside, ce qui garantit l’intégrité sans alourdir la chaîne.

Listing 6.2 – AnnouncementLog.sol — Structure et publication

```
struct Announcement {
    bytes32 contentHash; // SHA-256 du corps de l'annonce
    bytes32 group;
    string category; // "exam", "homework", "schedule"...
    uint256 timestamp;
    address publisher;
}

function publish(bytes32 contentHash, bytes32 group,
                string calldata category)
    external onlyProfessor
{
    require(contentHash != bytes32(0), "AnnouncementLog: empty hash");
    require(roleManager.isValidGroup(group), "invalid group");
    _announcements.push(Announcement({
        contentHash: contentHash, group: group,
        category: category, timestamp: block.timestamp,
        publisher: msg.sender
    }));
    emit AnnouncementPublished(...);
}
```

La fonction `getPage(offset, limit)` permet une pagination côté frontend pour éviter les limites de gas sur de grands tableaux.

6.1.3 AcknowledgmentLog — Preuve de lecture

Ce contrat enregistre la preuve cryptographique qu’un étudiant a pris connaissance d’une annonce. Il applique deux règles métier critiques avant tout enregistrement : la vérification de l’appartenance au groupe, et l’unicité de l’accusé de réception.

Listing 6.3 – AcknowledgmentLog.sol — Vérification du groupe

```
bytes32 studentGroup = roleManager.getGroup(msg.sender);
require(
    studentGroup == announcement.group
    || announcement.group == roleManager.GROUP_ALL(),
    "AcknowledgmentLog: announcement not for your group"
);
acknowledged[announcementId][msg.sender] = true;
emit Acknowledged(announcementId, msg.sender, block.timestamp);
```

Résultat — Non-répudiation garantie

Une fois enregistré, l'accusé de réception est permanent et publiquement vérifiable. L'horodatage de bloc constitue une preuve légalement opposable qu'un étudiant identifié par son adresse Ethereum a lu une annonce à un instant précis.

6.1.4 DocumentRegistry — Intégrité des fichiers

DocumentRegistry applique le concept de **hash pointer** : seule l'empreinte SHA-256 d'un fichier est stockée on-chain. Quiconque dispose du fichier original peut recalculer son hash et le comparer au registre pour vérifier que le contenu n'a pas été altéré. Un mécanisme anti-doublon (`hashExists`) empêche l'enregistrement du même fichier deux fois.

6.2 Backend Node.js

Le backend expose une API RESTful construite avec **Express v5** et **TypeScript**. Il joue le rôle d'intermédiaire entre le frontend et la blockchain, et héberge le pipeline RAG.

6.2.1 Service Blockchain (Viem)

La communication avec Ethereum repose sur **Viem**, une bibliothèque TypeScript type-safe. Le service expose deux clients : un `publicClient` pour la lecture (gratuite) et un `walletClient` pour les écritures signées.

Listing 6.4 – blockchain.ts — Initialisation des clients Viem

```
export const publicClient = createPublicClient({
    chain: hardhat,
    transport: http(process.env.RPC_URL),
});

// Encode les transactions pour signature MetaMask (frontend)
```

```
export function encodePublishTransaction(  
  contentHash: '0x${string}', group: string, category: string  
) : '0x${string}' {  
  return encodeFunctionData({  
    abi: ANNOUNCEMENT_LOG_ABI,  
    functionName: "publish",  
    args: [contentHash, keccak256(toBytes(group)), category],  
  });  
}
```

Point d'architecture — Signature MetaMask

Les transactions d'écriture (publication, accusé de réception) ne sont pas signées par le backend. Le backend encode les données de transaction et les retourne au frontend, qui les soumet à MetaMask pour signature par l'utilisateur. Ainsi, aucune clé privée utilisateur ne transite par le serveur.

6.2.2 Middleware d'authentification

Chaque route protégée reçoit l'adresse wallet de l'utilisateur via le header HTTP `x-wallet-address`. Le middleware interroge `RoleManager` on-chain pour vérifier le rôle avant de laisser passer la requête.

Listing 6.5 – `auth.ts` — Vérification du rôle on-chain

```
const ok = await publicClient.readContract({  
  address: CONTRACT_ADDRESSES.roleManager,  
  abi: ROLE_MANAGER_ABI,  
  functionName: "hasRole",  
  args: [roleKey, wallet],  
});  
if (!ok) return res.status(403).json({ error: "Forbidden" });  
next();
```

6.2.3 Pipeline RAG (Mistral AI)

Le service RAG indexe chaque annonce publiée en calculant son vecteur d'embedding via `mistral-embed`. À chaque question, il cherche les annonces les plus proches par similarité cosinus, filtrées par groupe, et fournit ce contexte au modèle `mistral-small-latest` pour la génération de la réponse.

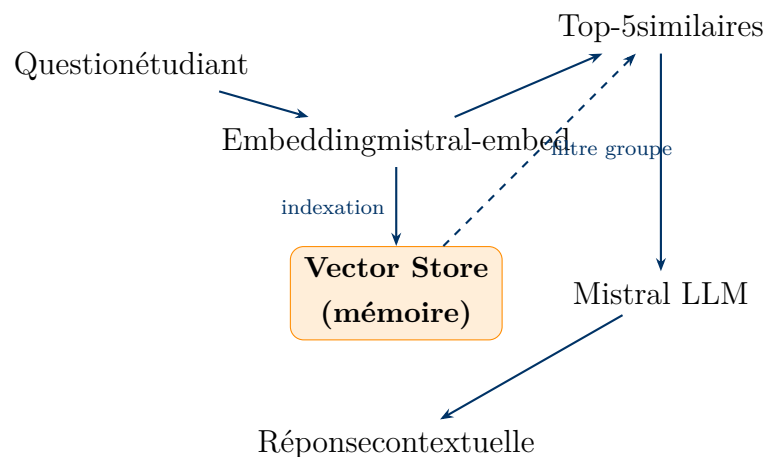


FIGURE 6.1 – Flux d’exécution du pipeline RAG intégré au backend

6.3 Bot Telegram

Le bot Telegram est implémenté avec **Telegraf** et démarre automatiquement avec le serveur Express. Il offre trois fonctionnalités principales :

1. **/wallet 0x...** — Lie un wallet Ethereum au compte Telegram et vérifie son rôle on-chain.
2. **/annonces** — Affiche les cinq dernières annonces récupérées depuis la blockchain.
3. **Question libre** — Déclenche le pipeline RAG et retourne une réponse contextualisée.

Résultat — Double interface utilisateur

Les étudiants peuvent accéder au système académique sans ouvrir de navigateur. Le bot Telegram partage exactement la même logique métier et le même pipeline RAG que l’interface web, via les mêmes services backend.

6.4 Déploiement des contrats

Le script `deploy.js` déploie les quatre contrats dans l’ordre de leurs dépendances : `RoleManager` en premier (les trois autres en dépendent), puis `AnnouncementLog`, `AcknowledgmentLog` et `DocumentRegistry`. À l’issue du déploiement, les groupes MF1 et MF2 sont enregistrés et les rôles de test sont assignés automatiquement.

Listing 6.6 – `deploy.js` — Ordre de déploiement

```

// 1. RoleManager (aucune dépendance)
const roleManager = await RoleManager.deploy();

// 2. AnnouncementLog (depend de RoleManager)

```

```
const announcementLog = await AnnouncementLog.deploy(roleManager.target);  
  
// 3. AcknowledgmentLog (depend des deux precedents)  
const acknowledgmentLog = await AcknowledgmentLog.deploy(  
  announcementLog.target, roleManager.target  
);
```

Les adresses générées sont ensuite copiées dans le fichier `.env` du backend, qui les utilise pour initialiser les clients Viem.

Chapitre 7

Interfaces utilisateur

7.1 Interface enseignant

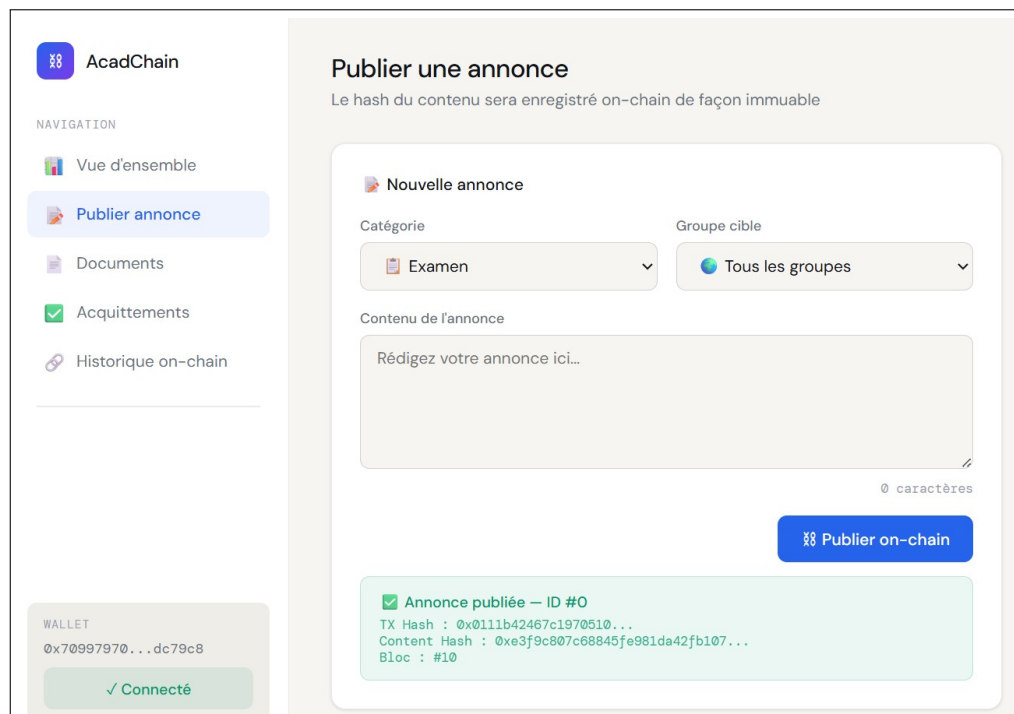


FIGURE 7.1 – Dashboard enseignant — Publication d’annonces et gestion des documents

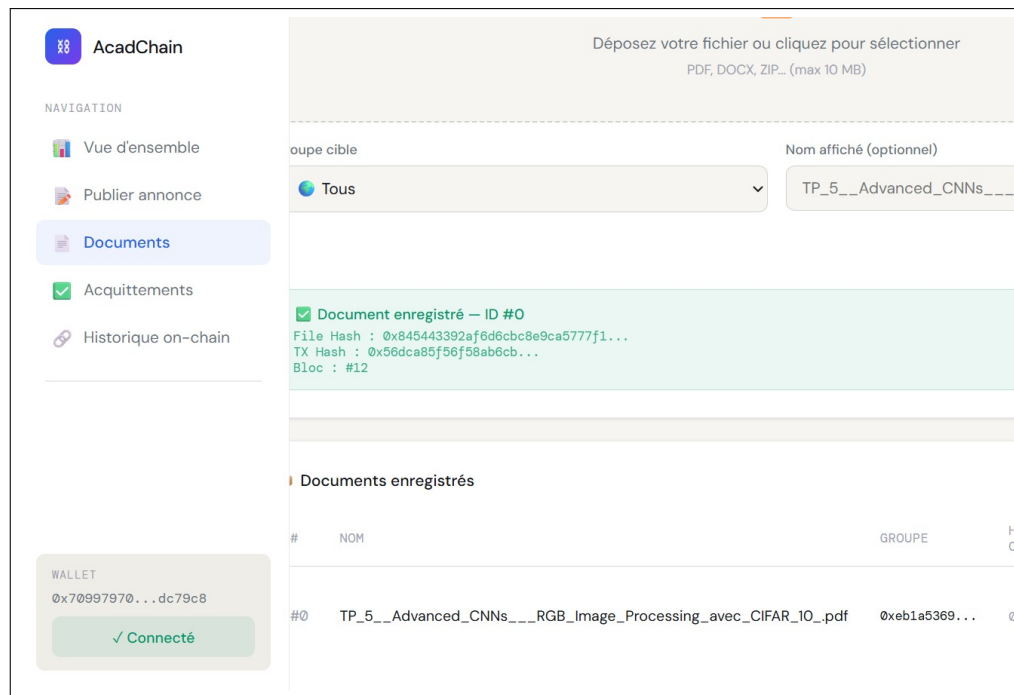


FIGURE 7.2 – Dashboard enseignant — Publication des documents

L'interface enseignant permet de publier des annonces et des documents en sélectionnant le groupe cible, la catégorie et le contenu. À la soumission, MetaMask ouvre une fenêtre de confirmation de transaction. Une fois la transaction confirmée sur la blockchain, l'annonce apparaît immédiatement dans la liste.

7.2 Interface étudiant

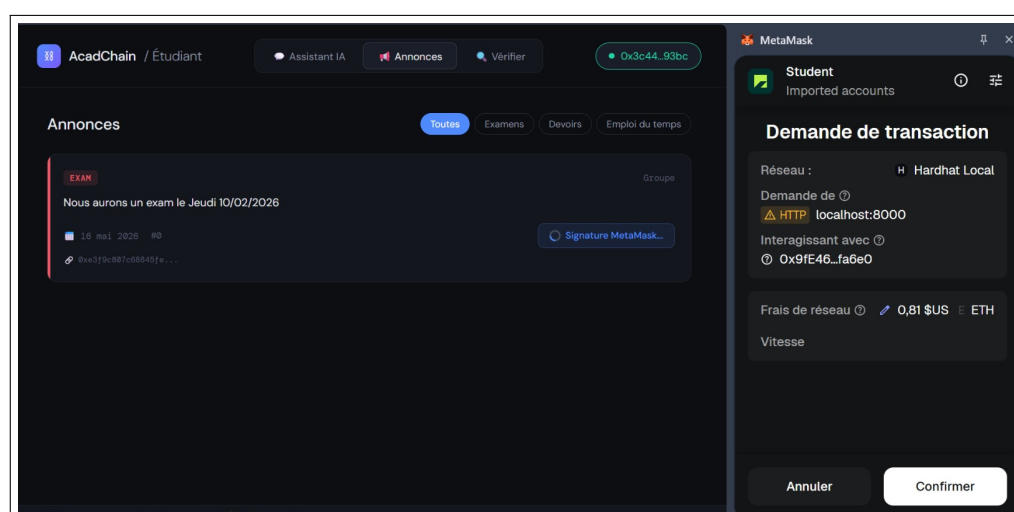


FIGURE 7.3 – Dashboard étudiant — Consultation des annonces de son groupe

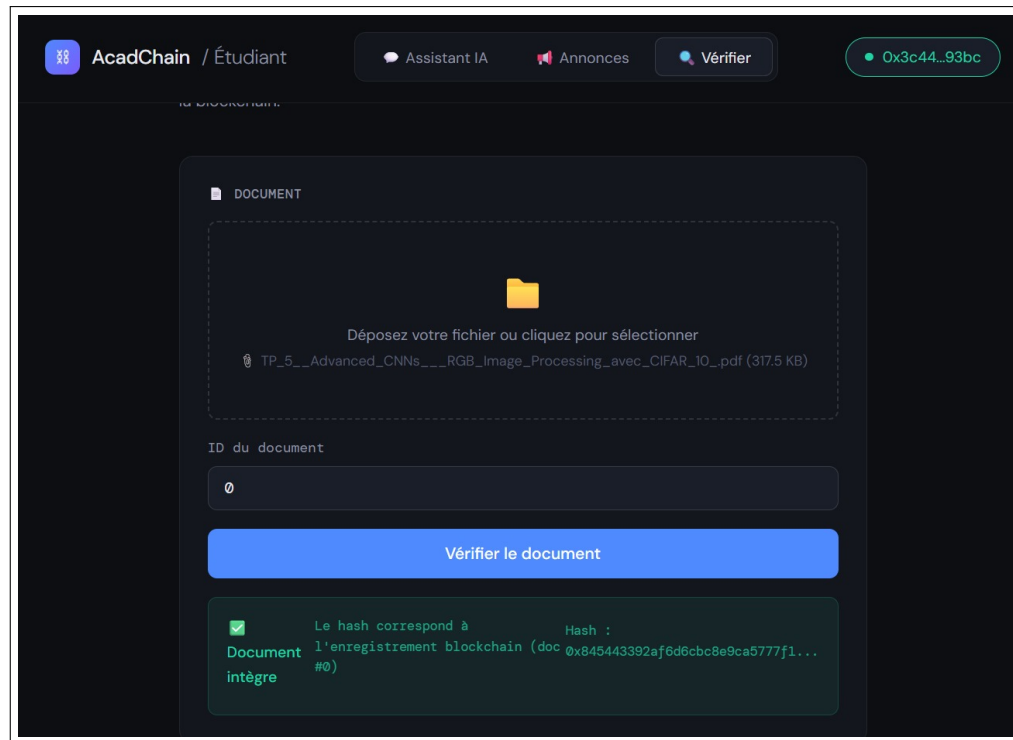


FIGURE 7.4 – Dashboard étudiant — Vérification des documents de son groupe

L'étudiant voit uniquement les annonces de son groupe. Pour chaque annonce, un bouton « Signer l'accusé de réception » déclenche une transaction MetaMask qui enregistre la preuve de lecture on-chain. Et pour chaque document une option vérifier le document pour s'assurer de l'intégrité de celui-ci.

7.3 Chatbot IA

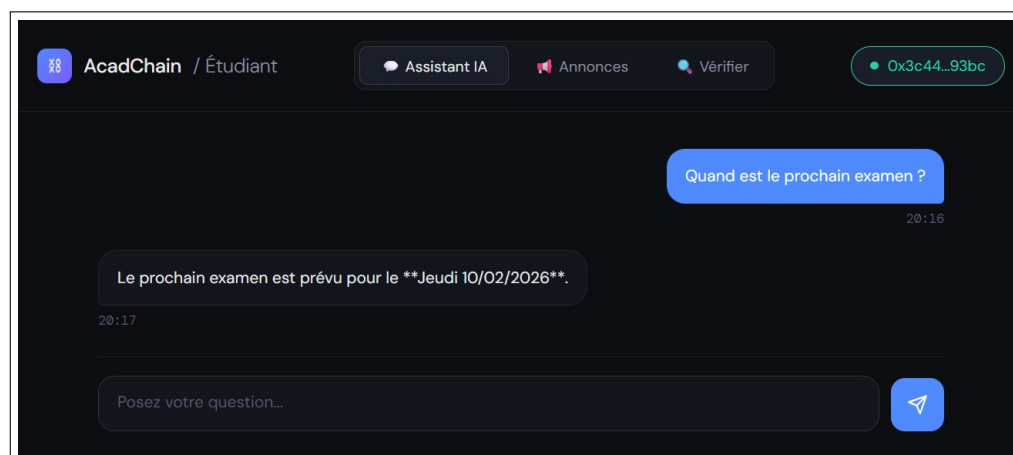


FIGURE 7.5 – Interface chatbot — Réponse contextuelle basée sur les annonces de l'étudiant

Le chatbot répond en français aux questions posées en langage naturel. Il filtre automatiquement les annonces par groupe et utilise le pipeline RAG pour fournir des réponses précises et contextualisées.

7.4 Confirmation MetaMask

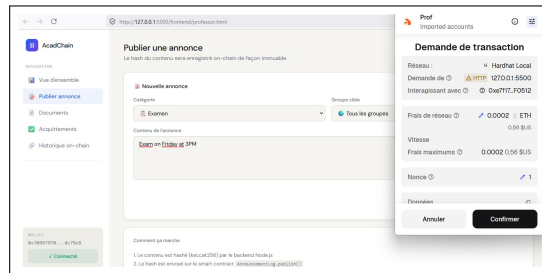


FIGURE 7.6 – Fenêtre de confirmation MetaMask lors d’une publication d’annonce

MetaMask affiche le contrat appelé, l’adresse de l’expéditeur, les frais de gas estimés et demande confirmation avant d’envoyer la transaction sur la blockchain.

7.5 Bot Telegram

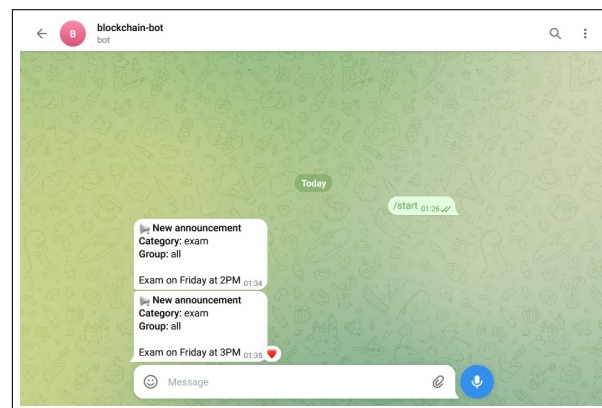


FIGURE 7.7 – Interaction avec le bot Telegram — Question et réponse IA

Le bot Telegram offre une interface conversationnelle mobile. Après liaison du wallet via `/wallet`, l’étudiant peut poser des questions directement depuis Telegram.

Chapitre 8

Sécurité

8.1 Immuabilité blockchain

Les données publiées sur la blockchain sont permanentes. Toute tentative de modification d'un bloc invalide l'ensemble de la chaîne suivante, rendant la falsification computationnellement impossible.

8.2 Intégrité des documents

Le hachage SHA-256 garantit l'intégrité des documents. Pour un fichier donné, tout changement d'un seul octet produit un hachage radicalement différent, détectable immédiatement par comparaison avec la valeur stockée on-chain.

8.3 Contrôle d'accès

Les smart contracts implémentent un contrôle d'accès basé sur les rôles (**RBAC**). Chaque fonction critique vérifie le rôle de `msg.sender` avant exécution via le contrat `RoleManager`. Aucune logique côté serveur ne peut contourner ces règles.

8.4 Non-répudiation

L'accusé de réception enregistré on-chain constitue une preuve cryptographique irréfutable qu'un étudiant (identifié par son adresse Ethereum) a pris connaissance d'une annonce à un instant précis (horodatage de bloc).

8.5 Authentification MetaMask

L'identité des utilisateurs est établie par leur capacité à signer des transactions avec leur clé privée. Le serveur ne stocke aucun mot de passe ni credential — il se contente de lire l'état on-chain.

Chapitre 9

Tests et validation

9.1 Tests des smart contracts

La suite de tests couvre les 4 contrats avec 29 cas de test utilisant **Hardhat** et **Chai** :

Contrat	Tests	Cas couverts
AnnouncementLog	9	Publication, vérification hash, groupe invalide, événements, accès non autorisé
AcknowledgmentLog	8	Lecture, double accusé, mauvais groupe, groupe «all» , ID invalide
DocumentRegistry	7	Enregistrement, doublon, intégrité, événements , vérification, accès non autorisé
RoleManager	5	Groupes, rôles, accès non autorisé , groupes invalides
Total	29	

TABLE 9.1 – Résumé de la suite de tests

```
1 it("Student cannot publish announcement", async function () {
2   const contentHash = hash("Fake announcement");
3
4   await expect(
5     announcementLog.connect(student).publish(
6       contentHash, GROUP_MF1, "exam"
7     )
8   ).to.be.revertedWith("AnnouncementLog: not a professor");
9 });
```

Listing 9.1 – Exemple de test — Étudiant ne peut pas publier

9.2 Tests backend

Les routes API ont été validées via des requêtes `curl` directes :

```
1 curl -X POST http://localhost:3001/api/annoncements \
2   -H "Content-Type: application/json" \
3   -H "x-wallet-address: 0x70997970..." \
4   -d '{"content":"Examen le 20 mai","category":"exam","group":"MF1"
5   # Reponse: { "success": true, "id": 0, "txHash": "0x4a40..." }
```

Listing 9.2 – Test de la route de publication d’annonce

9.3 Résultats



29/29

Tests passés

Tous les 29 tests unitaires passent avec succès. Les routes backend répondent correctement aux scénarios nominaux et aux cas d’erreur (rôle incorrect, double accusé, hash invalide).

Chapitre 10

Difficultés rencontrées

1. **Gestion des types `bytes32` vs `string`** : La représentation des groupes en `bytes32` (keccak256) optimise le gas mais nécessite une conversion cohérente entre le frontend, le backend et les contrats.
2. **Synchronisation des rôles** : Le backend utilise un wallet unique pour signer les transactions, ce qui crée un conflit quand plusieurs rôles sont nécessaires. La solution finale délègue la signature au frontend via MetaMask.
3. **Configuration réseau MetaMask** : Les utilisateurs doivent manuellement configurer le réseau Hardhat local (chainId 31337) dans MetaMask, ce qui représente une friction à l'onboarding.
4. **Persistance du vector store RAG** : Le store vectoriel est en mémoire et se réinitialise au redémarrage du backend, nécessitant une ré-indexation des annonces depuis la blockchain.

Chapitre 11

Perspectives d'amélioration

11.1 Stockage décentralisé avec IPFS

Intégrer **IPFS** (InterPlanetary File System) pour stocker les fichiers de manière décentralisée. Le smart contract **DocumentRegistry** stockerait le CID IPFS en plus du hash SHA-256, permettant la récupération du fichier depuis le réseau pair-à-pair.

11.2 Application mobile native

Développer une application **React Native** avec intégration du wallet via WalletConnect, offrant une expérience native iOS/Android supérieure au bot Telegram.

11.3 Déploiement multi-universités

Architecturer le système pour supporter plusieurs institutions, avec des namespaces d'administration séparés, permettant à chaque université de gérer ses propres rôles et annonces.

11.4 DAO universitaire

Implémenter un mécanisme de gouvernance décentralisée (**DAO**) permettant aux parties prenantes (enseignants, étudiants, administration) de voter sur les politiques académiques importantes.

11.5 Déploiement sur testnet public

Déployer les contrats sur **Sepolia** (testnet Ethereum) pour une validation en conditions réelles, puis sur le mainnet Ethereum ou une L2 (**Polygon**, **Arbitrum**) pour réduire les coûts de transaction.

Chapitre 12

Conclusion générale

Ce projet a permis de concevoir et d'implémenter un **Assistant Académique Décentralisé** fonctionnel, combinant les apports de la blockchain Ethereum avec les capacités de l'intelligence artificielle générative.

Les quatre smart contracts déployés garantissent l'immutabilité et la traçabilité des communications académiques, tandis que le pipeline RAG basé sur Mistral AI offre une interface conversationnelle intuitive filtrant les informations par groupe. L'intégration MetaMask assure une authentification cryptographique robuste, et le bot Telegram étend l'accessibilité aux plateformes mobiles.

D'un point de vue technique, ce projet a démontré la complémentarité entre la blockchain (couche de confiance immuable) et l'IA (couche d'accessibilité intelligente) : la blockchain constitue la **source de vérité**, tandis que l'IA en facilite l'accès et l'exploitation.

Les perspectives d'amélioration identifiées — intégration IPFS, persistance du vector store, déploiement multi-universités — offrent un cadre de développement stimulant pour une version production de la plateforme.

Ce projet illustre comment les technologies Web3 peuvent transformer des processus académiques traditionnels en systèmes transparents, résistants à la censure et vérifiables par tous.